

Less transient constexpr allocation (and more consteval relaxation)

Document #: P3032R2
Date: 2024-04-16
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Immediate-escalating expressions](#)

[2.2 Transient allocations](#)

[2.2.1 Constant Expression vs Core Constant Expression](#)

[2.2.2 Mutation](#)

[2.2.3 Is this actually to preserve constants?](#)

[3 Proposal](#)

[3.1 Incomplete Prior Wording](#)

[3.2 Wording](#)

[3.3 Feature-Test Macro](#)

[4 Acknowledgements](#)

[5 References](#)

§ 1 Revision History

[P3032R2]: [P3032R1] of this paper was actually on the Straw Polls page in Tokyo. Richard Smith pointed out one issue with the wording, and then Hubert Tong pointed out a much bigger issue with the wording - and it was pulled. This revision expands the description of the problem and addresses the wording issues. R1 also did not address the question of [escalating expressions](#), but this proposal expands on that too.

[P3032R1]: fixed wording, added feature-test macro.

§ 2 Introduction

C++20 introduced constexpr allocation, but in a limited form: any allocation must be deallocated during that constant evaluation.

The intent of the rule is that no `constexpr` allocation persists to runtime. For more on why we currently need to avoid that, see Jeff Snyder's [P1974R0] and also [P2670R1].

But the rule cited above does slightly more than prevent `constexpr` allocation to persist until runtime. The goal of this paper is to allow more examples of allocations that *do not* persist until runtime, that nevertheless are still rejected by the C++23 rules.

For the purposes of this paper, we'll consider the example of wanting to get the number of enumerators of a given enumeration. While the specific example is using reflection ([P2996R1]), there isn't anything particularly reflection-specific about the example - it just makes for a good example. All you need to know about reflection to understand the example is that `^E` gives you an object of type `std::meta::info` and that this function exists:

```
namespace std::meta {
    using info = /* ... */;
    constexpr vector<info> enumerators_of(info);
}
```

With that, let's go through several attempts at trying to get the *number* of enumerators of a given enumeration `E` as a constant:

	Attempt	Result
1	<pre>int main() { constexpr int r1 = enumerators_of(^E).size(); return r1; }</pre>	 . This one is valid - because <code>r1</code> is a <code>constexpr</code> variable, it's initializer starts a constant evaluation - which includes the entire expression. The temporary vector is destroyed at the end of that expression, so it doesn't persist outside of any constant evaluation.
2	<pre>constexpr int f2() { return enumerators_of(^E).size(); } int main() { constexpr int r2 = f2(); return r2; }</pre>	 . This one is invalid. The same idea about initializing <code>r2</code> as mentioned in the previous example is valid - but because <code>f2</code> is a <code>constexpr</code> function, the invocation of <code>enumerators_of(^E)</code> is not in an immediate function context - so it needs to be a constant expression on its own. That is, we start a new constant evaluation within the original one - but this constant evaluation is just the expression <code>enumerators_of(^E)</code> . It is not the full expression <code>enumerators_of(^E).size()</code> . As a result, the temporary vector returned by <code>enumerators_of(^E)</code> persists outside of its

		constant expression in order to invoke <code>.size()</code> on it, which is not allowed.
3	<pre> consteval int f3() { return enumerators_of(^E).size(); } int main() { constexpr int r3 = f(); return r3; } </pre>	✓. Both this row and the next row are subtle refinements of the second row that make it valid. The only difference between this and the previous row is that f2 was <code>constexpr</code> but f3 is <code>consteval</code>. This distinction matters, because now <code>enumerators_of(^E)</code> is no longer an immediate invocation - it is now in an immediate function context. As a result, the only thing that matters is that the entire expression <code>enumerators_of(^E).size()</code> is constant - and the temporary <code>vector<info></code> does not persist past that.
4	<pre> template<class E> constexpr int f4() { return enumerators_of(^E).size(); } int main() { constexpr int r4 = f4<E>(); return r4; } </pre>	✓. Here f4 is a <code>constexpr</code> function <i>template</i>, whereas f2 was a regular <code>constexpr</code> function. This matters because of [P2564R3] - the fact that <code>enumerators_of(^E).size()</code> isn't a constant expression now causes f4 to become a <code>consteval</code> function template - and thus we're in the same state that we were in f3: it's not <code>enumerators_of(^E)</code> that needs to be a core constant expression but rather all of <code>enumerators_of(^E).size()</code>.
5	<pre> consteval int f5() { constexpr auto es = enumerators_of(^E); return es.size(); } int main() { constexpr int r5 = f(); return r5; } </pre>	✗. Even though f5 is <code>consteval</code>, we are still explicitly starting a new constant evaluation within f5 by declaring <code>es</code> to be <code>constexpr</code>. That allocation persists past that declaration - <i>even though</i> it does not persist past f5, which by being <code>consteval</code> means that it does not persist until runtime.

Three of these rows are valid C++23 programs (modulo the fact that they're using reflection), but 2 and 5 are invalid - albeit for different reasons:

- in the 2nd row, we are required to consider `enumerators_of(^E)` as a constant expression all by itself - even if `enumerators_of(^E).size()` is definitely a constant expression.
- in the 5th row, we are required to consider `es` as a non-transient `constexpr` allocation - even though it definitely does not persist until runtime, and thus does not actually cause any of the problems that

non-transient `constexpr` allocation has to address.

§ 2.1 Immediate-escalating expressions

The wording in [P2564R3] introduced the term *immediate-escalating expression* in 7.7 [expr.const]:

- 17 An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either
- (17.1) — a potentially-evaluated *id-expression* that denotes an immediate function that is not a subexpression of an immediate invocation, or
 - (17.2) — an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.

In the second example:

```
constexpr int f2() {  
    return enumerators_of(^E).size();  
}
```

The expression `enumerators_of(^E)` is immediate-escalating - it is an immediate invocation (`enumerators_of` is a `constexpr` function) that is not a constant expression (because the temporary vector persists outside of this expression). This is what causes `f4` to become a `constexpr` function template.

But `enumerators_of(^E).size()` is not an immediate invocation (it simply has a subexpression that is an immediate invocation). However, if we were to define it as an immediate invocation - then it would not be an immediate-escalating expression anymore because it is actually a constant expression. And that would be enough to fix this example (as well as `f4` which would then itself not escalate to `constexpr` since it wouldn't need to).

Put differently: instead of escalating `enumerators_of(^E)` up to the nearest function, which we then try to make `constexpr` (and fail in the case of `f2` because `constexpr` functions are not *immediate-escalating*), we only need to escalate up to the nearest enclosing expression that could be a constant expression.

§ 2.2 Transient allocations

The wording in 7.7 [expr.const] for rejecting non-transient allocations rejects an expression `E` as being a core constant expressions if `E` evaluates:

- (5.18) — a *new-expression* ([expr.new]), unless the selected allocation function is a replaceable global allocation function ([new.delete.single], [new.delete.array]) and the allocated storage is deallocated within the evaluation of `E`;
- (5.20) — a call to an instance of `std::allocator<T>::allocate` ([allocator.members]), unless the allocated storage is deallocated within the evaluation of `E`;

That is - an allocation within E has to be transient to E. However, the rule we really want is that a constant allocation is transient to constant evaluation. In the fifth example:

```
constexpr int f5() {  
    constexpr auto es = enumerators_of(^E);  
    return es.size();  
}
```

The allocation in `enumerators_of(^E)` isn't transient to that expression, but it is definitely destroyed within `f5`, which is `constexpr`. That's important: if `f5` were `constexpr`, we'd have access to that allocation at runtime.

We can loosen the restriction such that an allocation within E must be deallocated within E or, if E is in an immediate function context, the end of that context. This would be the end of the `if constexpr { }` block or the end of the `constexpr` function. Such a loosening would allow `f5` above, but not if it's `constexpr`, and not if `es` were also declared `static`.

Now, allowing the declaration of `es` here has numerous other issues that are worth considering.

§ 2.2.1 Constant Expression vs Core Constant Expression

Right before plenary in Tokyo, Hubert Tong pointed out an important omission in the wording of this paper: it completely failed to solve the problem.

While the wording relaxes the rules for a *core constant expression*, it did not touch two other important rules: the definition of a *constant expression* and the requirements for the initialization of a `constexpr` variable.

Specifically, the existing rule in 9.2.6 [\[dcl.constexpr\]/6](#) requires that:

- 6 ... In any `constexpr` variable declaration, the full-expression of the initialization shall be a constant expression (`[expr.const]`).

where the term “constant expression” is defined in 7.7 [\[expr.const\]/14](#):

- 14 A *constant expression* is either a glvalue core constant expression that refers to an entity that is a permitted result of a constant expression (as defined below), or a prvalue core constant expression whose value satisfies the following constraints:
- (14.1) — if the value is an object of class type, each non-static data member of reference type refers to an entity that is a permitted result of a constant expression,
 - (14.2) — if the value is an object of scalar type, it does not have an indeterminate value (`[basic.indet]`),

- (14.3) — if the value is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object ([*expr.add*]), the address of a non-immediate function, or a null pointer value,
- (14.4) — if the value is of pointer-to-member-function type, it does not designate an immediate function, and
- (14.5) — if the value is an object of class or array type, each subobject satisfies these constraints for the value.

An entity is a *permitted result of a constant expression* if it is an object with static storage duration that either is not a temporary object or is a temporary object whose value satisfies the above constraints, or if it is a non-immediate function.

Attempting to declare a local `constexpr` variable to point to some allocation would violate this rule - we do not meet the requirements set out above.

However, before we go about trying to figure out how to relax the rule to allow allocations in automatic storage duration `constexpr` variables in immediate function contexts - Richard Smith pointed out another issue. This time not so much a *mistake* as a missed opportunity: allocations aren't the only example of results that are not permitted today but could be allowed if they're entirely within an immediate function context. For instance, taking a pointer to an immediate function. We have to prevent that from leaking to runtime, but if we're in a `constexpr` function - there's nothing to prevent:

```
constexpr void f() {}
constexpr void g() {
    // Ought to be valid, but isn't a constant expression, because
    // compile-time-only state escapes... into a compile-time-only
    // context.
    constexpr auto *p = f;
    p();
}
```

So now we have multiple ways in which we need to relax this rule. How do we go about doing it? We could be very precise in carving out specifically what we need - but this has a cost. We could fail to carve out enough, and have to keep refining the rule. But more importantly, the status quo is that we have two clear terms with clear usage: *core constant expression* and *constant expression*. Any attempt to introduce a third term in between them simply adds complexity. Is it worth doing so?

Let's say that instead we go all the way. If an automatic storage `constexpr` variable is declared in an immediate function context, its initializer does *not* have to be a constant expression - it only has to be a core constant expression. This allows the allocation examples that were the original motivation of the paper, and this allows the immediate function example that Richard brought up. It does also allow some weird cases:

```
constexpr int f(int n) {
    constexpr int &r = n; // ill-formed, becomes well-formed
```

```

    return r;
}

struct S {
    constexpr S() {}
    int i;
};

constexpr void g() {
    constexpr S s; // ill-formed, becomes well-formed
}

```

Both of these cases are... odd. They are rejected today for being an invalid permitted result (`n` doesn't have static storage duration) and indeterminate (`s.i` isn't initialized), respectively. And allowing them isn't great. But also any attempt to actually use `r` and `s` here in a constant expression won't work anyway. So we're not losing anything in terms of correctness.

I think on the whole it's better to stick with the simpler and easier-to-understand rule, even as it allows some odd and pointless code.

§ 2.2.2 Mutation

Consider the following example:

```

constexpr void f(int n) {
    constexpr int* a = new int(n); // ill-formed
    constexpr int* b = new int(1); // #1
    int c[*b];                      // #2
    ++*b;                           // #3
    int d[*b];                      // #4
    delete b;
}

```

The declaration of `a` is already ill-formed, so we don't have to do anything here.

Now, if the declaration of `c` is ill-formed (at #2), then we lose the point of declaring the local `constexpr` variable. We really do want it to be usable as a constant expression.

However, at the very least the declaration of `d` has to be ill-formed - this cannot be valid code that both declares an `int[1]` and an `int[2]`. There are two ways we can get there:

1. We can reject #1 as being insufficiently constant. This gets into the issues that `propconst` was trying to solve [[P1974R0](#)].
2. We can reject #3 for doing mutation.

It would be nice to not have to go full `propconst` just to solve this particular issue. We're entirely within the realm of the constant evaluator, so this problem is just simpler than having to deal with `constexpr`

allocation that leaks to runtime. And we very nearly already have wording to reject #3, that's 7.7 [expr.const]/5.16:

- (5.16) — a modification of an object ([expr.ass], [expr.post.incr], [expr.pre.incr]) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E;

It's just that here, `*b` did actually begin its lifetime within E (the call to f), so we don't violate this rule. We should simply extend this rule to be able to reject this case.

§ 2.2.3 Is this actually to preserve constants?

This was the original motivating example presented in this paper:

```
constexpr int f5() {  
    constexpr auto es = enumerators_of(^E);  
    return es.size();  
}
```

Here we don't actually need `es.size()` to be a *constant expression* - it is enough for it to be a *core constant expression*. So is it worth going through extra hoops to make it so that `es` is usable in constant expressions (i.e. have `es.size()` and `es[0]` both be constants) or is it sufficient for it to simply be a core constant expression?

I think it's worth it.

One problem is how to have constant data. Let's say you have a function that produces some data that you want to keep around at runtime as a lookup table:

```
constexpr auto get_useful_data() -> std::vector<Data>;
```

We don't have non-transient `constexpr` allocation, so we cannot declare a global `constexpr std::vector<Data>` to hold onto that. We need to hold it as an array. Specifically a `std::array<Data, N>`, since `Data[N]` isn't something you can return from a function. But how do you get N? One option is this:

```
constexpr auto get_useful_data_as_array() {  
    constexpr size_t N = get_useful_data().size();  
  
    // let's just assume for simplicity that Data is regular  
    std::array<Data, N> data;  
    std::ranges::copy(get_useful_data(), data.begin());  
    return data;  
}
```

This works, but relies on calling `get_useful_data()` twice. What if it's computationally expensive? Sure it's not expensive at *runtime*, but build times matter too. Attempting to avoid that double invocation leads to some elaborate solutions (e.g. [this one](#)). And it'd be nice if we could just avoid that entirely:

```
constexpr auto get_useful_data_as_array() {  
    // ill-formed today, proposed OK  
    constexpr std::vector<Data> v = get_useful_data();  
  
    // okay, because v is a constant  
    std::array<Data, v.size()> data;  
    std::ranges::copy(v, data.begin());  
    return data;  
}
```

This is arguably the obvious solution to this problem (well, aside from being able to have non-transient `constexpr` allocation). You can see a concrete example of this in [\[P2996R2\]](#):

```
template <typename S>  
constexpr auto get_layout() {  
    constexpr auto members = nonstatic_data_members_of(^S);  
    std::array<member_descriptor, members.size()> layout;  
    for (int i = 0; i < members.size(); ++i) {  
        layout[i] = { .offset=offset_of(members[i]),  
                     .size=size_of(members[i]) };  
    }  
    return layout;  
}
```

§ 3 Proposal

There are two separate potential changes here, that would each make one of the attempts above well-formed:

1. we could [escalate](#) expressions to larger expressions, so that `enumerators_of(^E).size()` becomes a constant expression, or
2. we could extend the notation of [transient allocation](#) to include the full immediate context instead of just the constant evaluation

The second of these is straightforward to word and provides a lot of value - since now particularly in the context of reflection you can declare a `constexpr` `vector<info>` inside a `constexpr` function and use those contents as a constant expression.

The first of these is a little more complicated and doesn't provide as much value. It's a limitation that is fairly easy to work around: either declare a local `constexpr` variable, or change the function to be `constexpr` or a template. However, it is pretty annoying to have to do so - and it would be nice if we kept pushing `constexpr` evaluation more in the direction of "it just works." Following [\[P2564R3\]](#), the rules

here are already pretty complicated - but the advantage of that is that users simply don't have to learn them if more things just work.

This paper proposes solving both problems. That is, all five examples in the [intro](#) will be valid.

§ 3.1 Incomplete Prior Wording

Also pointed out by Richard, the original wording changing 7.7 [\[expr.const\]/5](#) as follows:

(5.18) a *new-expression* ([*expr.new*]), unless the selected allocation function is a replaceable global allocation function ([*new.delete.single*], [*new.delete.array*]) and the allocated storage is deallocated **either** within the evaluation of *E* **or, if *E* is in an immediate function context, within that context**;

Richard pointed out this example, asking if it's valid:

```
constexpr void f(bool b) {  
    constexpr int *p = new int;  
    if (b) delete p;  
}
```

Noting that it's impossible to tell - it depends on *b*, which the constant evaluator does not know. Instead he suggests this wording:

(5.18) a *new-expression* ([*expr.new*]), unless the selected allocation function is a replaceable global allocation function ([*new.delete.single*], [*new.delete.array*]) and **either** the allocated storage is deallocated within the evaluation of *E*, **or *E* is in an immediate function context**;

He points out that the actual call to *f* still has to be a constant expression, and so this leak rule still applies there. Neither leaks-to-runtime nor compile-time leaks are possible. So this wording change is more correct.

Richard also points out that this allows this nonsensical function, but if you can't observe a leak, does it really leak?

```
constexpr void f() {  
    if (false) { constexpr int *p = new int; }  
}
```

§ 3.2 Wording

Change 7.7 [\[expr.const\]/5](#):

5 An expression E is a *core constant expression* unless the evaluation of E, following the rules of the abstract machine ([intro.execution]), would evaluate one of the following:

(5.1) — [...]

(5.16) — a modification of an object ([expr.ass], [expr.post.incr], [expr.pre.incr]) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E;

(5.16b) — a modification of an object ([expr.ass], [expr.post.incr], [expr.pre.incr]) whose lifetime began within the evaluation of the initializer for a constexpr variable V, unless E occurs within the initialization or destruction of V or of a temporary object whose lifetime is extended to that of V;

[Example 1:

```
constexpr int f(int n) {
    constexpr int* p = new int(1); // #1
    ++n;          // ok, lifetime of n began within E
    ++*p;         // error: modification of object whose lifetime began
                  within
                  // initializer of constexpr variable at #1
    delete p; // ok
    constexpr int q = []{ // #2
        int i = 0;
        ++i; // ok: modification of an object whose lifetime begin
              within E
              // this E occurs within the initialization of constexpr
              variable
              // declared at #2
        return i;
    }();
    return n + q;
}
```

— end example]

(5.17) — [...]

(5.18) — a *new-expression* ([expr.new]), unless the selected allocation function is a replaceable global allocation function ([new.delete.single], [new.delete.array]) and either E is in an immediate function context or the allocated storage is deallocated within the evaluation of E;

[Example 2:

```
constexpr int f() {
    constexpr int* i = new int(1); // error: allocation is neither
    deallocated within this
    return *i;                    // evaluation nor within an
    immediate function context
}

constexpr int o() {
    constexpr int* n = new int(21); // ok, #1
}
```

```

    int a = *n;
    delete n;                // #2
    return a;
}

constexpr int e() {
    constexpr int* r = new int(2022); // ok, #3
    return *r;
}

static_assert(o() == 21);      // ok, because allocation at #1 is
                             deallocated at #2
static_assert(e() == 2022);    // error: allocation at #3 is not
                             deallocated

```

— *end example*]

- (5.19) — a *delete-expression* ([`expr.delete`]), unless it deallocates a region of storage allocated within the evaluation of *E*;
- (5.20) — a call to an instance of `std::allocator<T>::allocate` ([`allocator.members`]), unless either *E* is in an immediate function context or the allocated storage is deallocated within the evaluation of *E*;
- (5.21) — a call to an instance of `std::allocator<T>::deallocate` ([`allocator.members`]), unless it deallocates a region of storage allocated within the evaluation of *E*;
- (5.22) — [...]

Change 7.7 [expr.const]/16-18:

...¹⁶std An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

- .1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function,
- .2) — it is a subexpression of a manifestly constant-evaluated expression or conversion, or
- .3) — its enclosing statement is enclosed ([`stmt.pre`]) by the compound-statement of a `constexpr` if statement ([`stmt.if`]).

An invocation is an *immediate invocation* if it is a potentially-evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An aggregate initialization is an immediate invocation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression. An expression is an immediate invocation if it is a constant expression, has a subexpression that would otherwise be immediate-escalating (see below), and does not have a subexpression that would also meet these criteria.

- 17 An expression or conversion is *immediate-escalating* if it is not initially in an immediate function context and it is either
 - .1) — a potentially-evaluated *id-expression* that denotes an immediate function that is not a subexpression of an immediate invocation, or

- .2) — an immediate invocation that is not a constant expression and is not a subexpression of an immediate invocation.

18 An *immediate-escalating* function is

- .1) — the call operator of a lambda that is not declared with the `constexpr` specifier,
- .2) — a defaulted special member function that is not declared with the `constexpr` specifier, or
- .3) — a function that results from the instantiation of a templated entity defined with the `constexpr` specifier.

An immediate-escalating expression shall appear only in an immediate-escalating function.

19 An immediate function is a function or constructor that is

- .1) — declared with the `constexpr` specifier, or
- .2) — an immediate-escalating function *F* whose function body contains an immediate-escalating expression *E* such that *E*'s innermost enclosing non-block scope is *F*'s function parameter scope.

[*Note 11*: Default member initializers used to initialize a base or member subobject ([`class.base.init`]) are considered to be part of the function body ([`dcl.fct.def.general`]). — *end note*]

[*Example 9*:

```
constexpr int id(int i) { return i; }
constexpr char id(char c) { return c; }

template<class T>
constexpr int f(T t) {
    return t + id(t);
}

auto a = &f<char>;           // OK, f<char> is not an immediate function
auto b = &f<int>;            // error: f<int> is an immediate function

static_assert(f(3) == 6);    // OK

template<class T>
constexpr int g(T t) {      // g<int> is not an immediate function
    return t + id(42);      // because id(42) is already a constant
}

template<class T, class F>
constexpr bool is_not(T t, F f) {
    return not f(t);
}

constexpr bool is_even(int i) { return i % 2 == 0; }

static_assert(is_not(5, is_even));    // OK

int x = 0;
```

```

template<class T>
constexpr T h(T t = id(x)) {    // h<int> is not an immediate function
                                // id(x) is not evaluated when parsing the default
                                argument ([dcl.fct.default], [temp.inst])
    return t;
}

template<class T>
constexpr T hh() {              // hh<int> is an immediate function because of the
    invocation                  // of the immediate function id in the default
    return h<T>();              // argument of h<int>
}

int i = hh<int>();              // error: hh<int>() is an immediate-escalating
                                expression
                                // outside of an immediate-escalating function

struct A {
    int x;
    int y = id(x);
};

template<class T>
constexpr int k(int) {          // k<int> is not an immediate function because A(42)
    is a                        // constant expression and thus not immediate-
    return A(42).y;             // escalating
}

+ consteval std::vector<int> get_data();
+
+ constexpr int get_size1() {
+     constexpr auto v = get_data(); // error: get_data() is an immediate-escalating
+         expression
+                                     // outside of an immediate-escalating function
+     return v.size();
+ }
+
+ constexpr int get_size2() {
+     return get_data().size();      // OK, get_data().size() is an immediate invocation
+         that is
+                                     // a constant expression
+ }

```

— *end example*]

Change 9.2.6 [[dcl.constexpr](#)]/6:

- 6 A `constexpr` specifier used in an object declaration declares the object as `const`. Such an object shall have literal type and shall be initialized. In any `constexpr` variable declaration, **either**
 - (6.1) — the full-expression of the initialization shall be a constant expression (`[expr.const]`), **or**

- (6.2) — the variable shall have automatic storage duration, be declared within an immediate function context, and the full-expression of the initialization shall be a core constant expression ([[expr.const](#)]).

Except for an automatic storage duration variable declared in an immediate function context, a `constexpr` variable that is an object, as well as any temporary to which a `constexpr` reference is bound, shall have constant destruction.

[*Example 4*:

```
struct pixel {
    int x, y;
};
constexpr pixel ur = { 1294, 1024 };    // OK
constexpr pixel origin;                // error: initializer missing

+ constexpr int f() {
+     constexpr pixel* q = new pixel{3, 4}; // ok
+     int result = q->x + q->y;
+     delete q;
+     return result;
+ }
+
+ constexpr void g() {
+     constexpr pixel* p = new pixel{1, 2}; // error: not a constant
+         expression
+     delete p;
+     constexpr auto pf = f; // error: not a constant expression
+ }
+
+ constexpr int h() {
+     constexpr auto pf = f; // ok
+     return pf();
+ }
```

— *end example*]

§ 3.3 Feature-Test Macro

Bump the value of `__cpp_constexpr` in 15.11 [[cpp.predefined](#)]:

```
- __cpp_constexpr 202306L
+ __cpp_constexpr 2024XXL
```

§ 4 Acknowledgements

Thank you to Peter Dimov for being Peter Dimov and coming up with all of these examples.

Thank you to Hubert Tong for noticing that the wording was wrong and Richard Smith for helping to fix it. Thanks to Jason Merrill for help with phrasing the immediate-escalating wording.

§ 5 References

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.

<https://wg21.link/p1974r0>

[P2564R3] Barry Revzin. 2022-11-11. consteval needs to propagate up.

<https://wg21.link/p2564r3>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.

<https://wg21.link/p2670r1>

[P2996R1] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2023-12-18. Reflection for C++26.

<https://wg21.link/p2996r1>

[P2996R2] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-02-15. Reflection for C++26.

<https://wg21.link/p2996r2>

[P3032R1] Barry Revzin. 2023-03-21. Less transient constexpr allocation.

<https://wg21.link/p3032r1>

[P3032R2] Barry Revzin. 2024-04-16. Less transient constexpr allocation (and more consteval relaxation).

<https://wg21.link/p3032r2>